

Spring Boot

Input/Output

Parametri in input

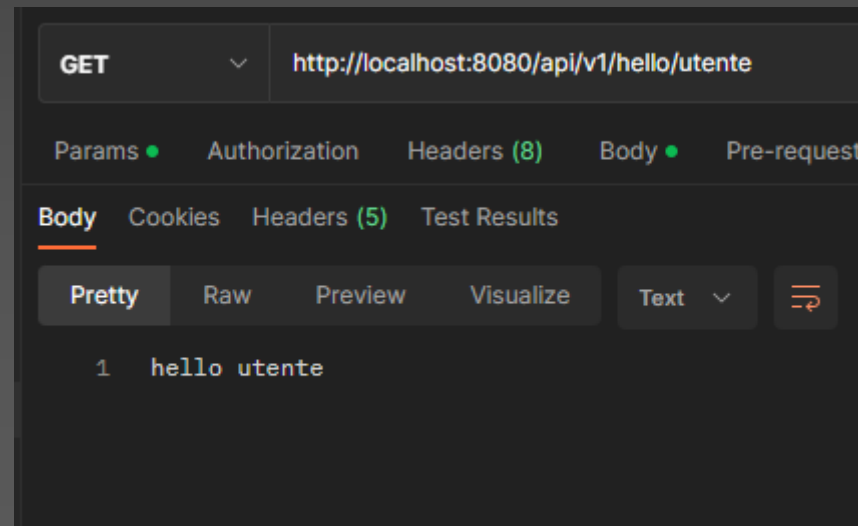
- Per eseguire delle operazioni, un web service necessita dei dati input.
- Oltre alle rappresentazioni di risorse è possibile anche passare dei parametri nella richiesta ad una risorsa.
- Tipicamente nelle operazioni di lettura.
- Due tipi:
 - URI path parameters
 - Query parameters
- Entrambi estraggono dati dall'URI

URI path parameters

➤ @PathVariable

```
@RestController
@RequestMapping(value = "/api/v1")
public class HelloController{

    @GetMapping("/hello/{key}")
    public String hello(@PathVariable("key")
String nome){
        return "hello "+nome;
    }
}
```



Query parameters



➤ @RequestParam

```
@RestController
@RequestMapping(value = "/api/v1")
public class HelloController{

    @GetMapping("/hello")
    public String hello(@RequestParam("nome")
String nome){
        return "hello "+nome;
    }
}
```

GET ⌵ http://localhost:8080/api/v1/hello?nome=utente

Params • Authorization Headers (6) Body Pre-request Script Tests Settings

Query Params

	KEY	VALUE
☒	nome	utente
	Key	Value

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize Text ⌵

1 hello utente



Esercizio

Creare un controller REST il quale contiene un'endpoint GET che prende in input nome, cognome, titolo corso.

Deve restituire una stringa che simula l'accettazione di uno studente ad un corso:

"Gentile, <nome_input> <cognome_input>, la informiamo della sua ammissione al corso <titolo_corso_input> in data <data_attuale>".

Eserzio da fare sia con URI path parameters che con Query parameters.

Esercizio

Creare un controller REST il quale contiene un'endpoint GET che prende in input numero1, numero2, operatore(addizione,sottrazione,moltiplicazione,divisione) e restituisce il risultato calcolato. In di caso errore dobbiamo restituire un messaggio di errore.

Esercizio da fare sia con URI path parameters che con Query parameters.

Esercizio



Creare un controller REST il quale contiene un'endpoint GET che prende in input un'email.

Tale email deve essere validata(non deve essere null, vuota e deve contenere la '@'). In caso di email non valida deve essere restituito un messaggio di errore. In caso email valida, bisogna verificare l'esistenza di quest'ultima all'interno di una lista di stringhe che contiene le email registrate. Se l'email non è presente nella lista restituire un messaggio di esito negativo durante la ricerca, se invece l'email è presente restituire l'email.

Eserzio da fare sia con URI path parameters che con Query parameters.



Annotazioni metodi HTTP

HTTP Method	@RequestMapping	Spring 4.3 Annotation
GET	@RequestMapping(method=RequestMethod.GET)	@GetMapping
POST	@RequestMapping(method=RequestMethod.POST)	@PostMapping
DELETE	@RequestMapping(method=RequestMethod.DELETE)	@DeleteMapping
PUT	@RequestMapping(method=RequestMethod.PUT)	@PutMapping

- GET: lettura risorsa
- POST: creazione risorsa
- PUT: modifica risorsa
- DELETE: eliminazione risorsa

Annotazioni metodi HTTP

- Tutte le annotazioni dei metodi HTTP sono internamente annotate con `@RequestMapping`.
- Tale annotazione contiene degli attributi che si possono usare anche sulle nuove annotazioni.
 - value, path
 - name
 - headers
 - produces
 - consumes

```
@PostMapping(value =  
    "/automobile/aggiungi",  
              consumes =  
MediaTypes.APPLICATION_JSON_VALUE,  
              produces =  
MediaTypes.APPLICATION_JSON_VALUE)
```

Conversione Rappresentazioni

- La conversione da oggetto JAVA a JSON e viceversa, viene svolta in automatico.
- Il bean che ha il compito di svolgere le conversioni implementa l'interfaccia `HttpMessageConverter<T>`.
- Esistono diverse implementazioni concrete di tale interfaccia. Ognuna pensata per un tipo di conversione diversa.
- Di default questo bean converte gli oggetti JAVA in JSON e viceversa. Ma è possibile riconfigurarli per aggiungere altre tipologie di conversione.(es: XML ecc..)

Conversione in JSON



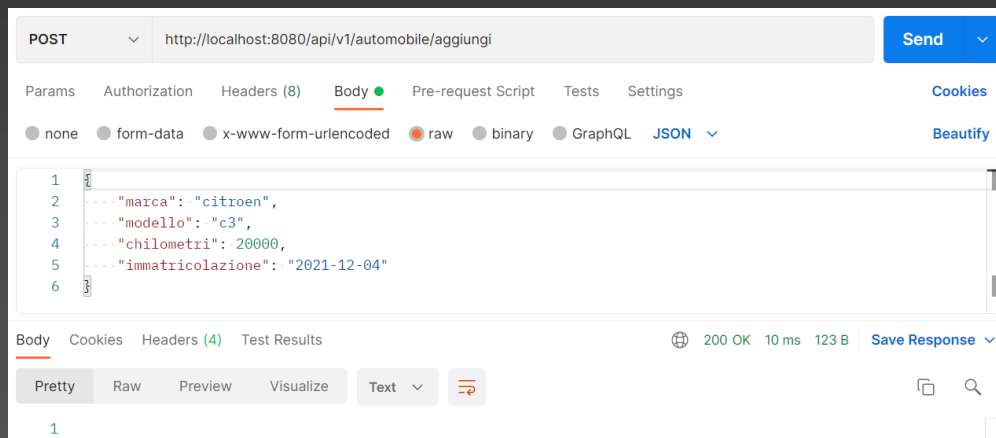
```
@GetMapping("/automobile")
public Automobile getAutomobile(){
    return new
Automobile(1,"fiat","panda",400000,25, 2,1999);
}
```

```
{
  "id": 1,
  "marca": "fiat",
  "modello": "panda",
  "chilometri": 400000,
  "immatricolazione": "1999-02-25"
}
```



Conversione Rappresentazioni

- `@RequestBody` comunica a spring di effettuare la conversione. Il tipo di conversione da applicare dipende dall'header «content-type» della richiesta HTTP.



```
Automobile
[id=0,
 marca=citroen,
 modello=c3,
 chilometri=20000,
 immatricolazione=2021-12-04]
```

```
@PostMapping("/automobile/aggiungi")
public void addAutomobile(@RequestBody Automobile auto){
    System.out.println(auto);
}
```

Esercizio



Creare un controller REST che abbia un endpoint GET che prende nome, cognome, eta come parametri in input, e restituisce un'oggetto di tipo Persona che contiene i dati presi in input.



Esercizio



Creare una classe Database che è un Singleton, al suo interno avremo una lista di studenti, ogni studente avrà come attributi int id, String email, String nome, String cognome, double mediaVoti

Popolare con qualche studente la lista

Creare un controller REST chiamato "CorsoController".

Creare un endpoint GET che restituisce tutti gli studenti del database.

-Creare un endpoint GET che prende come parametro in input l'email e restituisce lo studente che ha quell'email. Se l'email non è valida o lo studente non esiste restituire null.

-Creare un endpoint GET che restituisce lo studente che ha la media dei voti più alta.

-Creare un endpoint GET che restituisce lo studente che ha la media dei voti più bassa.

-Creare un endpoint POST che riceve una rappresentazione di una risorsa in JSON. I dati contenuti nel JSON devono essere validati(nessuno deve essere null o vuoto e la media voti non deve essere 0). In caso di validazione negativa allora restituire un esito negativo. In caso di validazione positiva aggiungere lo studente alla lista degli studenti e restituire un esito positivo.

-Creare un endpoint PUT che prende come parametro in input l'id dello studente da modificare e la nuova email che tale studente dovrà avere. In caso l'id dello studente non esista o la nuova email non sia valida, allora restituire un esito negativo. In caso i dati siano validi effettuare la modifica dello studente nella lista e restituire un esito positivo.

-Creare un endpoint DELETE che prende come parametro in input l'id dello studente da eliminare dalla lista. Se l'id non esiste allora restituire un esito negativo, se l'id esiste allora eliminare lo studente e restituire un esito positivo.



Risposta HTTP

- Fornire al client risposte HTTP che contengono:
 - Codice di stato
 - Corpo della risposta
 - Headers aggiuntivi(opzionale)
- Per raggiungere questo obiettivo ci sono diverse modalità.
 - `ResponseEntity<>`
 - Altre..

ResponseEntity<T>

```
@PostMapping("/hello/{nome}")  
public ResponseEntity<String> hello(@PathVariable("nome")  
String nome){  
    return new ResponseEntity<>("ciao "+nome, HttpStatus.OK);  
}
```

```
@PostMapping("/hello")  
public ResponseEntity<String> hello(@RequestParam("nome") String nome){  
    if(nome==null||nome.isBlank()) return  
ResponseEntity.badRequest().build();  
    return ResponseEntity.status(HttpStatus.OK).body("Ciao "+nome);  
}
```

Esercizio



Ripetere l'esercitazione precedente ma con `ResponseEntity<>`.

-In aggiunta, modificare l'endpoint GET che riceve in input l'email e restituisce lo studente dell'esercizio precedente. Dopo aver superato validazioni e controlli, verificare l'esistenza dello studente nella lista. In tal caso, restituire lo studente, codice 200 e un header HTTP custom aggiuntivo chiamato "Authorization". Il valore di tale header deve essere l'iniziale dell'email, l'iniziale del nome, l'iniziale del cognome concatenate ad un numero di 4 cifre generato randomicamente.

In caso di lettura avvenuto con successo -> code 200

In caso di aggiunta avvenuta con successo -> code 201

In caso di modifica o eliminazione avvenuta con successo -> code 204

In caso di dati in input non validi -> code 400

In caso di risorsa non trovata -> 404

